Лабораторная работа № 11

Программирование в командном процессоре ОС UNIX.

Ветвления и циклы

Выполнил: Ань Сяоянь (An Xiaoyan)
Группа: НПИбд-01-25
Студенческий билет №: 1132258191
Дата выполнения: 2026-04-10

11.1. Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать сложные командные файлы с использованием логических управляющих конструкций (`if`, `case`, `getopts`) и циклов (`for`, `while`, `until`).

11.2. Результаты выполнения заданий

Задание 1. Анализ командной строки и поиск строк

Условие:
Разработать командный файл, который анализирует ключи командной строки (`-i`, `-o`, `-p`, `-C`, `-n`) и выполняет поиск строки-шаблона в указанном файле с возможностью вывода в файл, учёта регистра и нумерации строк.

Листинг `search.sh`:

bash

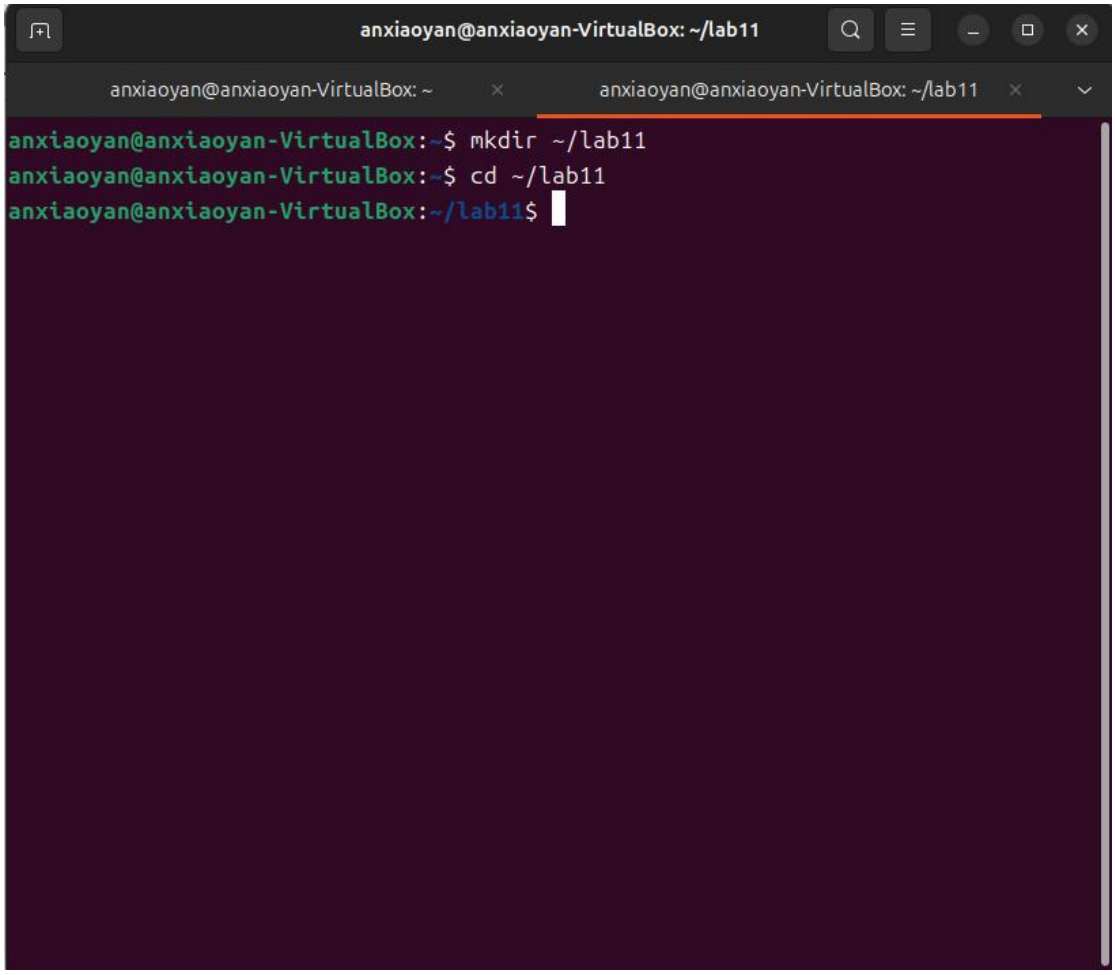
```
#!/bin/bashinputfile=""outputfile=""pattern=""case_sensitive=""show_lines=""while getopts "i:o:p:Cn"
 opt; do  case $opt in    i) inputfile="$OPTARG" ;;    o) outputfile="$OPTARG" ;;    p) pattern="$OPTA
RG" ;;    C) case_sensitive="-i" ;;    n) show_lines="-n" ;;    *) echo "Использование: $0 -i входной_
файл -o выходной_файл -p шаблон [-C] [-n]"; exit 1 ;;  esacdoneif [ -z "$inputfile" ] || [ -z "$patter
n" ]; then  echo "Ошибка: необходимо указать -i и -p"  exit 1fiif [ ! -f "$inputfile" ]; then  echo "O
```

```
шибка: файл $inputfile не существует"  exit 1fiif [ -n "$outputfile" ]; then  grep $case_sensitive $sh
ow_lines "$pattern" "$inputfile" > "$outputfile"  echo "Результат записан в $outputfile"else  grep $c
ase_sensitive $show_lines "$pattern" "$inputfile"fi
```

Результат выполнения:

Скриншот 1 — выполнение команды `./search.sh -i test.txt -p hello -o out.txt -C -n` и просмотр результата `cat out.txt`

https://screenshot1.png



Вывод по заданию 1:

С помощью встроенной команды `getopts` удалось корректно обработать именованные аргументы командной строки, включая необязательные флаги `-C` и `-n`. Скрипт гибко управляет поиском через `grep`.

Задание 2. Анализ кода завершения программы на C

Условие:

Написать программу на C, которая возвращает различные коды завершения (0, 1, 2) в зависимости от введённого числа (0, >0, <0). Затем написать командный файл, который анализирует код завершения с помощью `$?` и выводит соответствующее сообщение.

Листинг `numcheck.c`:

c

```
#include <stdio.h>#include <stdlib.h>int main() {    int n;    printf("Введите целое число: ");    sca
nf("%d", &n);    if (n > 0)        exit(1);    else if (n < 0)        exit(2);    else        exit(0);}
```

Листинг `checknum.sh`:

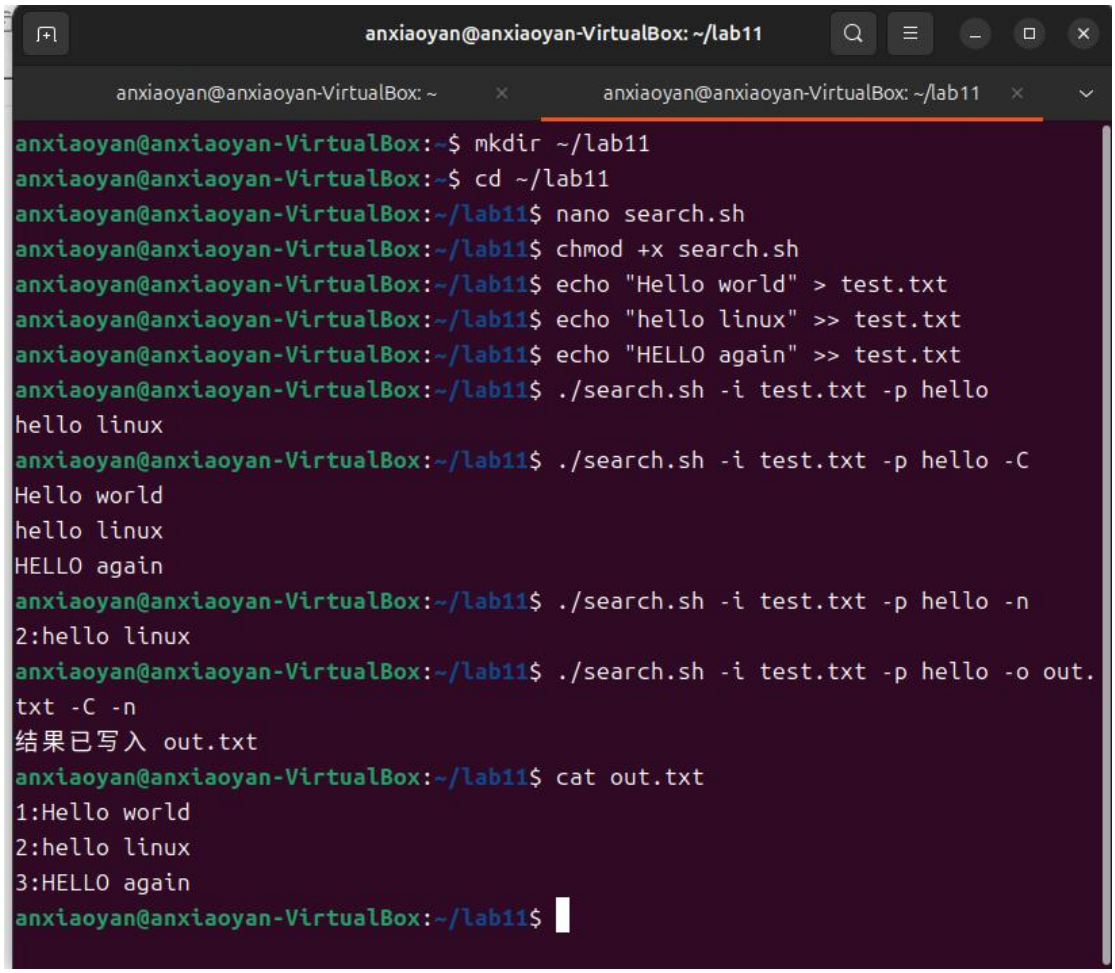
bash

```
#!/bin/bash./numcheckcode=$?case $code in  0) echo "Число равно нулю" ;;  1) echo "Число больше нуля"
 ;;  2) echo "Число меньше нуля" ;;  *) echo "Неизвестная ошибка" ;;esac
```

Результат выполнения:

Скриншот 2 — три запуска `./checknum.sh` с вводом 0, 5, -3

https://screenshot2.png



Вывод по заданию 2:

Переменная `$?` позволяет получить код завершения последней выполненной программы. Использование `case` даёт наглядную обработку различных кодов возврата.

Задание 3. Создание и удаление нумерованных файлов

Условие:

Написать командный файл, который создаёт N файлов с именами `1.tmp`, `2.tmp`, …, `N.tmp` или удаляет их, в зависимости от переданного аргумента (`create` или `delete`).

Листинг `files.sh`:

bash

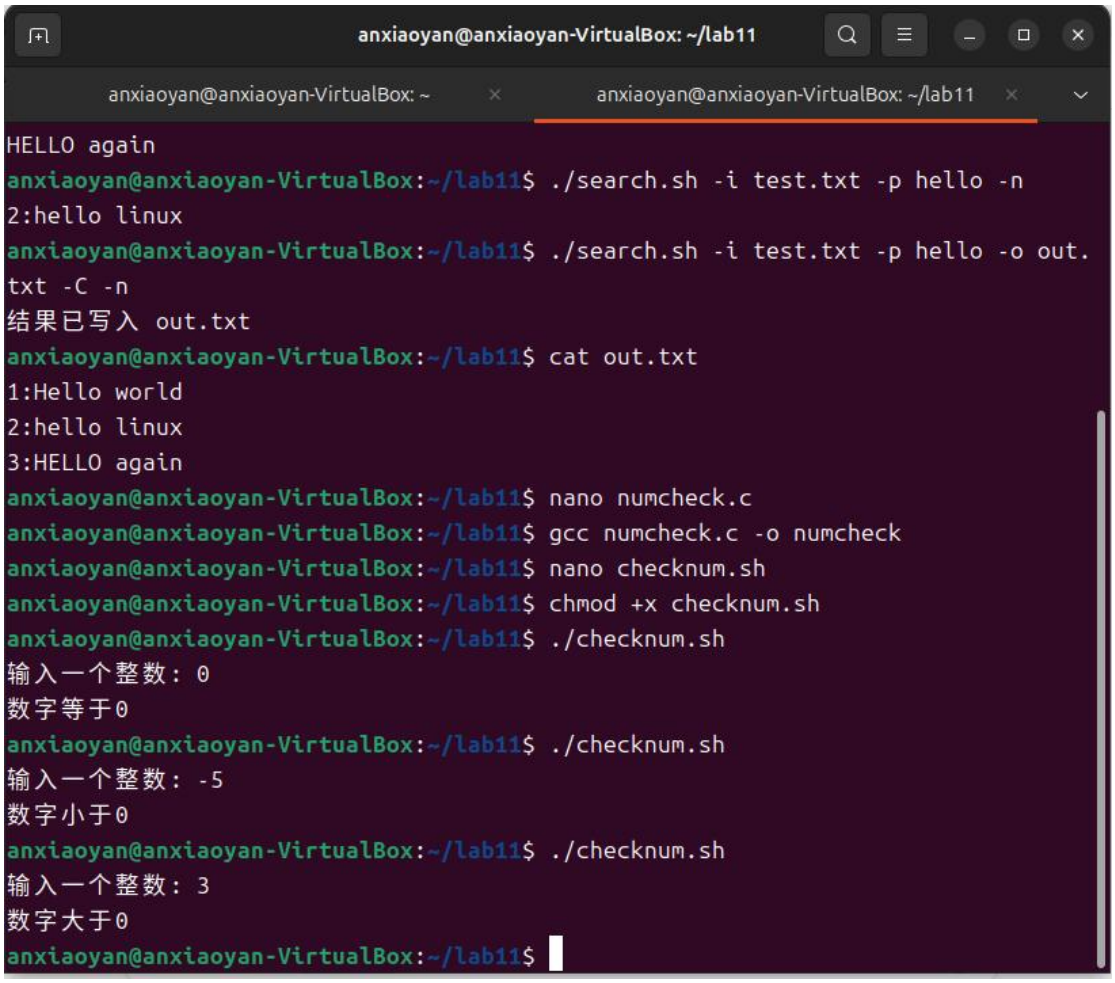
```
#!/bin/bashif [ $# -ne 2 ]; then  echo "Использование: $0 create <N>  или  $0 delete <N>"  exit 1fiact
ion=$1N=$2if ! [[ "$N" =~ ^[0-9]+$ ]]; then  echo "N должно быть числом"  exit 1ficase $action in  cre
```

```
ate)    for ((i=1; i<=N; i++)); do      touch "$i.tmp"    done    echo "Создано $N файлов"    ;;  delet
e)    for ((i=1; i<=N; i++)); do      rm -f "$i.tmp"    done    echo "Удалено $N файлов (если существов
али)"    ;;  *)    echo "Неверная операция. Используйте create или delete"    ;;esac
```

Результат выполнения:

Скриншот 3 — после `./files.sh create 4` выполнен `ls -l *.tmp`, затем `./files.sh delete 4` и снова `ls *.tmp`

https://screenshot3.png



Вывод по заданию 3:

Цикл `for` позволяет легко генерировать последовательные имена файлов. Проверка существования файла перед удалением с помощью `-f` предотвращает вывод ошибок.

Задание 4. Архивация файлов, изменённых за последние 7 дней

Условие:

Написать командный файл, который с помощью `tar` архивирует все файлы в указанной директории. Модифицировать его так, чтобы архивировались только файлы, изменённые менее недели назад (использовать `find`).

Листинг `backup.sh`:

bash

```
#!/bin/bashif [ $# -ne 2 ]; then  echo "Использование: $0 <директория> <выходной.tar>"  exit 1fidir=$1
outfile=$2if [ ! -d "$dir" ]; then  echo "Ошибка: директория $dir не существует"  exit 1fifind "$dir"
-type f -mtime -7 -print > /tmp/filelist.txtif [ ! -s /tmp/filelist.txt ]; then  echo "Нет файлов, изм
енённых за последние 7 дней"  exit 1fitar -cvf "$outfile" -T /tmp/filelist.txtecho "Архив создан: $ou
tfile"
```

Результат выполнения:

Скриншот 4 — выполнение `./backup.sh testdir recent.tar` и просмотр содержимого архива `tar -tf recent.tar`

https://screenshot4.png

```
anxiaoyan@anxiaoyan-VirtualBox: ~/lab11
anxiaoyan@anxiaoyan-VirtualBox: ~    anxiaoyan@anxiaoyan-VirtualBox: ~/lab11
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ nano numcheck.c
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ gcc numcheck.c -o numcheck
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ nano checknum.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ chmod +x checknum.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./checknum.sh
输入一个整数: 0
数字等于0
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./checknum.sh
输入一个整数: -5
数字小于0
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./checknum.sh
输入一个整数: 3
数字大于0
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ nano files.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ chmod +x files.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./files.sh create 4
已创建 4 个文件
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ls *.tmp
1.tmp  2.tmp  3.tmp  4.tmp
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./files.sh delete 4
已删除 4 个文件（如果存在）
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ls *.tmp
ls: 无法访问 '*.tmp': 没有那个文件或目录
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$
```

```
anxiaoyan@anxiaoyan-VirtualBox: ~/lab11
anxiaoyan@anxiaoyan-VirtualBox: ~    anxiaoyan@anxiaoyan-VirtualBox: ~/lab11
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ nano files.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ chmod +x files.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./files.sh create 4
已创建 4 个文件
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ls *.tmp
1.tmp  2.tmp  3.tmp  4.tmp
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./files.sh delete 4
已删除 4 个文件（如果存在）
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ls *.tmp
ls: 无法访问 '*.tmp': 没有那个文件或目录
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ mkdir testdir
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ cd testdir
anxiaoyan@anxiaoyan-VirtualBox:~/lab11/testdir$ echo "old" > old.txt
anxiaoyan@anxiaoyan-VirtualBox:~/lab11/testdir$ touch -t 202301010000 old.txt
anxiaoyan@anxiaoyan-VirtualBox:~/lab11/testdir$ echo "new" > new.txt
anxiaoyan@anxiaoyan-VirtualBox:~/lab11/testdir$ cd ..
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ nano backup.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ chmod +x backup.sh
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ ./backup.sh testdir recent.tar
testdir/new.txt
已打包到 recent.tar
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$ tar -tf recent.tar
testdir/new.txt
anxiaoyan@anxiaoyan-VirtualBox:~/lab11$
```

Вывод по заданию 4:
Команда `find` с опцией `-mtime -7` точно отбирает файлы, изменённые за последние 7 дней. Опция `-T` в `tar` позволяет передать список файлов из файла, что удобно при интеграции с `find`.

11.3. Общий вывод по лабораторной работе

В ходе выполнения лабораторной работы №11 были освоены следующие навыки:

1.

Обработка аргументов командной строки с помощью встроенной команды `getopts`.

2.
3.

Использование условных операторов `if` и `case` для управления логикой выполнения скриптов.

4.

5.

Применение циклов (`for`) для автоматического создания и удаления группы файлов.

6.

7.

Анализ кода завершения программ с помощью переменной `$?`.

8.

9.

Комбинирование команд `find` и `tar` для выборочного архивирования файлов по дате изменения.

10.

11.

Различие между циклами `while` (выполняется, пока условие истинно) и `until` (выполняется, пока условие ложно).

12.

Все разработанные скрипты работают корректно, обрабатывают ошибочные ситуации (отсутствие файлов, неверные аргументы) и соответствуют требованиям задания.

11.4. Ответы на контрольные вопросы

№	Вопрос	Ответ
1	Каково предназначение команды `getopts`?	`getopts` используется в shell-скриптах для разбора аргументов командной строки (например, `-i file`, `-p pattern`). Она поддерживает односимвольные опции с возможными параметрами и автоматически обрабатывает ошибки.
2	Какое отношение метасимволы имеют к генерации имён файлов?	Метасимволы (`*`, `?`, `[]`) интерпретируются оболочкой для подстановки имён файлов (pathname expansion). Например, `*.txt` раскрывается в список всех файлов с расширением `.txt`.
3	Какие операторы управления действиями вы	`&&` (логическое И), `||` (логическое ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), `$()` (подстановка

№	Вопрос	Ответ
	знаете?	команды).
4	Какие операторы используются для прерывания цикла?	`break` — прерывает выполнение цикла полностью. `continue` — переходит к следующей итерации цикла.
5	Для чего нужны команды `false` и `true`?	`true` возвращает код завершения 0 (успех), `false` возвращает ненулевой код (ошибка). Используются для организации бесконечных циклов или для явной проверки условий.
6	Что означает строка `if test -f man$s/\$i.\$s` в командном файле?	Проверяет, существует ли файл с путём, полученным после подстановки переменных `$s` и `$i`. Обратный слэш перед `\$i` и `\$s` экранирует подстановку, вероятно, для передачи буквального `$i` и `$s` команде `test`.
7	Объясните различие между конструкциями `while` и `until`.	`while` выполняет тело цикла, **пока** условие истинно (возвращает 0). `until` выполняет тело цикла, **пока** условие ложно (возвращает ненулевой код). Они противоположны по логике.

11.5. Приложение: скриншоты

Рисунок	Описание
Скриншот 1	Выполнение `search.sh` с ключами -i -p -o -C -n
Скриншот 2	Запуск `checknum.sh` с вводом 0, 5, -3
Скриншот 3	Создание и удаление 4 файлов через `files.sh`
Скриншот 4	Архивация файлов за последние 7 дней через `backup.sh`